- The nesting at the top-level of code interpreted by `Kernel#load` is empty unless the `load`call receives a true value as second argument, in which case a newly created anonymous module is pushed by Ruby.

It is interesting to observe that blocks do not modify the stack. In particular the blocks that may be passed to `Class.new` and `Module.new` do not get the class or module being defined pushed to their nesting. That's one of the differences between defining classes and modules in one way or another.

2.2 Class and Module Definitions are Constant Assignments

Let's suppose the following snippet creates a class (rather than reopening it):

```
class C
end
```

Ruby creates a constant `C` in `Object` and stores in that constant a class object. The name of the class instance is "C", a string, named after the constant.
That is,

```
class Project < ActiveRecord::Base
end
```

performs a constant assignment equivalent to

```
Project = Class.new(ActiveRecord::Base)
```

including setting the name of the class as a side-effect:

```
Project.name # => "Project"
```

Constant assignment has a special rule to make that happen: if the object being assigned is an anonymous class or module, Ruby sets the object's name to the name of the constant.

From then on, what happens to the constant and the instance does not matter. For example, the constant could be deleted, the class object could be assigned to a different constant, be stored in no constant anymore, etc. Once the name is set, it doesn't change.

Similarly, module creation using the `module` keyword as in

```
module Admin
end
```

performs a constant assignment equivalent to

```
Admin = Module.new
```

including setting the name as a side-effect:

```
Admin.name # => "Admin"
```

The execution context of a block passed to `Class.new` or `Module.new` is not entirely equivalent to the one of the body of the definitions using the `class` and `module` keywords. But both idioms result in the same constant assignment.

Thus, when one informally says "the `String` class", that really means: the class object stored in the constant called "String" in the class object stored in the `Object` constant. `String` is otherwise an ordinary Ruby constant and everything related to constants such as resolution algorithms applies to it.
Likewise, in the controller

```
class PostsController < ApplicationController
  def index
```